

7.1

I would replace the comments as such:

```
// Repeating indefinitely until one of two conditions is met
```

```
// If the remainder is 0, we've found the GCD, so return it.
```

```
// Otherwise, find the GCD of the smaller factor and the remainder
```

7.2

Programmers either write AS they code, or AFTER they code, but in both situations, the comments reflect what the code currently does, not what it's supposed to do. For future documentation purposes, future programmers reading your work should already know what the code does from the code itself, and understand the purpose from your comments.

7.4

I would first assert that both values are numbers, and that neither is equal to 0. Then, I would verify that a is larger than b.

7.5

If both values aren't numbers, I would have to throw an exception immediately because we can't even begin to go through with the calculation. I would return an error to the user stating, "one or both of the entered values are not numbers.". If either is equal to zero, I would also throw an exception: "values cannot be zero". If a isn't larger than b, I would return an error saying, "a must be larger than b".

7.7

1. Take my car keys and leave the house to enter my garage.
2. Use my keys to unlock my Lamborghini, and enter (the doors open up by the way).
3. Turn on the car, reverse out of the garage, and close the garage door with the fob on my car keys.
4. Use your phone and enter in the location of the nearest Ralph's.
5. Follow the directions on the phone accordingly.
6. Park away from the other cars so no one dings any part of the car.

Assumptions:

- The person already knows how to drive a car
- The person is familiar with traffic laws of the region
- The person is familiar with mobile devices
- The person is able-bodied and mentally capable of driving

8.1

testRelativelyPrime():

```
// testing proper handling of values that are too large
assertRaises( ValueError, isRelativelyPrime(1000001, 1))
assertRaises( ValueError, isRelativelyPrime(1000001, -1000001))

// testing any values with 1 and -1
assertTrue( isRelativelyPrime(10, 1))
assertTrue( isRelativelyPrime(1000000, 1))

assertTrue( isRelativelyPrime(10, -1))
```

```
assertTrue( isRelativelyPrime(1000000, -1)
```

```
// testing good values
```

```
assertTrue( isRelativelyPrime(10, 9))
```

```
assertFalse( isRelativelyPrime(10, 8))
```

8.3

- a. Since I know exactly how the method works, such as its value limits, I conducted white-box testing.
- b. If I didn't know the inner workings of the function, Black-Box testing would be possible. After learning a bit more about the internals, Gray-Box testing becomes possible. Since `isRelativelyPrime()` is a fairly basic function, White-Box testing is very straightforward. It wouldn't really make sense or be reasonable to conduct exhaustive testing though, testing 1000000 * 1000000 combinations of numbers when we know what the answers are already.

8.5

I implemented it in Python. The first bug I found was forgetting to test for non-integer values such as strings and long values, because Python doesn't enforce typed inputs on its own. Silly me!

Since my testing reminded me of a crucial design choice for the Python language, I'd say that's a net positive for my learning.

8.9

Exhaustive testing would have to count as black-box testing, because you are putting every single possible input into the method without knowledge of or care for attacking specific components of the method's internals.

8.11

We can estimate that we have $(5 * 4 * 5) / 3$ or 33 bugs or so in the code still.

8.12

If the two testers don't find any bugs in common, then I would assume we simply multiply the two numbers found, meaning our lower bound of bugs in common would be 1 for the sake of the equation. To be safe, we can assume that the Lincoln index underestimates the true number of bugs, so it can be the lower bound.